Sorting, Selection, and Randomized Analysis — Exercises

A03-E01 Expectations add even when events depend

Roll two independent fair dice and let X count sixes. Use indicators to find $E[X]$ and $\Pr(X = 1)$, identifying exactly where independence is used. Repeat when the second die always copies the first. Give the full distribution of X in each model and check its total probability and expectation.

A03-E02 Fixed-point expectation and dependent events

Place $n \geq 2$ labeled name cards uniformly at random into n positions. Let F count cards in their own positions. Find $E[F]$, then the probability that two specified cards are both fixed; compare it with the product of marginal probabilities. Give the distribution for $n = 2$ and handle $n = 0, 1$ separately.

A03-E03 Reverse Fisher–Yates: trace six complete paths

Use a zero-based array of distinct records $[a, b, c]$. For $i = 2, 1$, choose j_i uniformly from $\{0, \ldots, i\}$ **conditional on all prior choices**, then swap positions i, j_i. Give the intermediate array and output for all six paths (j_2, j_1). Prove probability $1/n!$ for each full permutation in the general algorithm; uniform position marginals alone are insufficient.

A03-E04 Random splits: work through the adjacent difference

A call of size $n \geq 2$ prints n stars, chooses p uniformly from $\{1, \ldots, n-1\}$, and recurses on sizes p and $n - p$. Size one prints only one exclamation mark. Each choice remains conditionally uniform given the past. Find the leaf count in every execution and expected total stars S_n. Subtract adjacent recurrence equations explicitly, solve the closed form, and tabulate means for $n = 1..5$.

A03-E05 Exactly two hires: condition on the best position

Candidates have distinct ranks, with larger better, and arrive in a uniform random permutation of size $n \geq 1$. Hire the first, then only candidates better than everyone previously seen. For total hires X, find probabilities of one, two, and n hires. Derive the two-hire formula by disjoint cases, evaluate at $n = 6$, and discuss overlapping event names when $n = 1, 2$.

A03-E06 Uniform position marginals can hide a biased shuffle

First, uniformly choose a right cyclic shift $s \in \{0, 1, 2, 3\}$ of $[a, b, c, d]$. Prove that every record has probability $1/4$ at each position, yet the full permutation is not uniform.

Second, process $i = 0, 1, 2$ on $[a, b, c]$. Independently choose j_i uniformly from all three positions and swap $A[i], A[j_i]$. Use path counts to prove bias, then tabulate counts for each output. Contrast the fixed-prefix invariant of forward Fisher–Yates.

A03-E07 Priority ties: condition on acceptance correctly

Start with $[a, b]$, draw independent uniform priorities $U_a, U_b \in \{1, 2, 3, 4\}$, and sort increasingly, preserving original order on ties. Find both output probabilities. Then reject the whole pair on a tie and draw fresh independent priorities each round. Prove uniform eventual output and almost-sure termination, and find expected rounds. Is there a fixed worst-case round count?

A03-E08 Nonuniform birthdays: pair counts versus event probability

People independently choose among $m \geq 1$ dates with probabilities $p_r \geq 0$ summing to one. Find a pair's collision probability and the expected number of colliding pairs among $k \geq 2$ people. Prove uniform probabilities minimize the pair probability. For four probabilities $[1/2, 1/4, 1/8, 1/8]$, find both the expected pair count and the exact probability of any collision among three people. List all four distinct-date subsets and their weights.

A03-E09 Four fixed throws versus waiting for collection

Independently throw four labeled balls uniformly into three bins. Find expected empty bins, expected bins containing exactly one ball, and the probability all bins are occupied. In a separate experiment, keep throwing from empty bins until collection; find $E[T]$ by tabulating success probabilities and mean waits by the number of occupied bins. Distinguish four fixed throws from a random stopping time.

A03-E10 Overlapping windows: count events with tail states

Flip eight independent fair coins. Let X count all-H windows among the six length-three windows and let L be the longest H run. Find $E[X]$. Then count prefixes without HHH by trailing H lengths zero, one, and two; tabulate states from step zero through eight and find $\Pr(L \geq 3)$. Explain why the six window events cannot be treated as independent.

A03-E11 Seven-person online choice: optimize and trace a failure

Seven distinct candidate ranks arrive in a uniform random order; larger is better. For integer $1 \leq k \leq 6$, reject the first k, then choose the first record better than everyone previously seen. If none is chosen before the end, choose the last candidate. Decisions are irrevocable; success means choosing the global best. Derive P_k, use adjacent differences to find the best integer threshold, list all six probabilities, and trace $[3, 1, 6, 2, 4, 7, 5]$.

A03-E12 Quicksort: count distinct and equal-key inputs separately

Six distinct keys satisfy $z_1 < \cdots < z_6$. Choose each pivot uniformly and use Lomuto, with one key comparison per non-pivot record. Find pair-comparison probabilities for z_2, z_5 and z_1, z_6, then sum expected total comparisons by rank distance. Repeat for six equal keys with a $\leq$ two-way partition; state the general-n difference and the three-way remedy.

A03-E13 Simulate six outcomes: rejection versus remainder

Using only independent fair bits, generate an exactly uniform integer in $1..6$. Prove the eventual distribution and almost-sure termination, find expected bit cost, and discuss a fixed worst-case bound. Compare directly returning `z mod 6 + 1` for a three-bit integer $z \in \{0, \ldots, 7\}$ by listing each output's preimages and probability.

A03-E14 Unknown biased coin: fairness and nonuniform cost bounds

A coin independently shows H with an unknown fixed probability $0 < p < 1$ each toss. Produce a fair bit without knowing p. Find acceptance probability, eventual output probabilities, termination probability, and expected toss count. Compare costs at $p = 1/2, 1/10, 1/100$, discuss a uniform constant bound over all p, and explain whether equal marginals alone suffice without independence.

A03-E15 Repair the root and record every swap

Use one-based $A = [4, 9, 8, 3, 7, 6, 5]$ with active size seven. Both child subtrees are valid max-heaps. Run the chapter's iterative `MAX-HEAPIFY`, stopping on equal keys. For every step, record the current index, valid child keys, selected child index, swap decision, and full array. Explain termination and verify all final parent-child edges.

A03-E16 Heap height sum: nodes do not all have root height

A complete binary tree with fifteen records is perfect. Heights count edges. Tabulate node counts and contributions by height h, then compute $\sum_i height(i)$. Prove that the height sum for a nonempty complete tree of size $n \geq 1$ is below n, and explain why per-call constant overhead must also be counted.

A03-E17 Increase at position eight: write, sift, and stop

Given valid max-heap $A = [20, 15, 18, 7, 12, 9, 17, 2]$ with one-based indices, increase position eight to sixteen. Record the array after writing and each swap, current and parent indices, and comparison results. Explain why only upward repair is needed and whether the old key two remains in the heap.

A03-E18 An incorrect parent formula rejects a valid heap

For a one-based array, a proposed validator checks $A[\lceil c/2 \rceil] \geq A[c]$ for every child $c = 2..n$. Test it on $A = [10, 1, 9]$, identify the first erroneous child index, give the correct one-based and zero-based formulas, and explain why checking every parent-child edge suffices.

A03-E19 Stopping on equal keys does not make heapsort stable

Records are $2a, 2b, 1c$ in that order. Letters identify records; compare only numeric keys. Use ascending heapsort with heapify stopping on equality. Trace the built heap, root-to-end exchange, repair, and final output. Decide stability and explain how adding original sequence numbers changes the comparison key.

A03-E20 Indexed deletion chooses direction from the parent edge

Start with valid max-heap $A = [50, 20, 45, 10, 15, 40, 44, 5, 9, 12, 14, 35, 39, 42, 43]$. Independently restart from this original array for deleting position four and deleting the root. Replace with the last record and list every effective prefix and exchanged indices.

Prove why a replacement exceeding its parent needs only upward repair and otherwise needs only downward repair. Address deleting the old final position, deleting from an empty queue, and handle-map updates. All positions are one-based.

A03-E21 A minimum pivot: trace every Lomuto test

Use one-based $A = [7, 3, 8, 3, 6, 2]$. Fix $x = A[6] = 2$, initialize $i = 0$, and scan $j = 1..5$. On $A[j] \le x$, increment i and swap $A[i], A[j]$. Finally swap $A[i + 1], A[6]$. Tabulate every test, i, and the full array; give final pivot index q and both recursive intervals. Explain the imbalance.

A03-E22 Bounded split ratios: account for every level

For every subproblem of size $m \ge 2$, partition costs at most cm. The two disjoint child sizes sum to $m - 1$, and each is at most $9m/10$. Stop at sizes zero and one. Prove $O(n \lg n)$ total cost by bounding path depth, total size at one level, and leaf cost. Must every split be exactly $1 : 9$?

A03-E23 Four distinct keys: when are endpoints compared?

Four distinct keys satisfy $z_1 < z_2 < z_3 < z_4$ in any initial order. Choose each pivot uniformly from its current interval; Lomuto compares it once with every other key there. Condition on the first pivot to find the probability that z_1, z_4 are directly compared. Distinguish remaining together from already comparing with each other.

A03-E24 Equal keys: why random pivots cannot rescue this split

All $n \ge 0$ keys equal five. Choose each pivot uniformly, move it to the end, and run Lomuto with a $\le$ test. Counting only scan key comparisons, find the exact total and list round costs for $n = 5$. State the three-way alternative's recursive intervals and cost on this input.

A03-E25 Recurse on the smaller side: stack versus work

After partitioning, quicksort recurses only on the smaller side and handles the larger side with a while loop in the current call. Sizes at most one return immediately. Prove worst-case stack space $O(\lg(n+1))$ for nonempty input. On a path whose pivot is always maximum, count key comparisons and simultaneous call frames. Distinguish sequential loop iterations from simultaneous stack depth.

A03-E26 Negative and repeated keys: what the invariant guarantees

Run one-based Lomuto on $A = [-1, 4, 0, 4, 2]$, using final pivot two and a $\leq$ test. Record initial i, every j, the test, and the full array after any swap, including final pivot placement. Give q and both side intervals. Use a counterexample to decide whether partition sorts each side.

A03-E27 Outside pivots, conditioning, and the comparison model

Seven distinct keys are $1..7$ in any initial order. Choose each pivot uniformly from its current subarray and use this chapter's Lomuto partition.

1. Find the probability that keys two and five directly compare. Condition on the first pivot from $\{2, 3, 4, 5\}$ and explain why the denominator is not seven.
2. Recompute conditional probabilities if the first pivot of the whole execution is seven, or three.
3. If a three-way partition uses successive $<$ and $>$ tests, is $2(n+1)H_n - 4n$ its exact Boolean-comparison count? Also contrast the interval and return contracts here with the Data Structures sorting chapter.

A03-E28 Four-key lower bound: necessity versus construction

Four labeled records have distinct keys, and order information comes only from key comparisons. Use a decision tree to prove that at least five comparisons are necessary in the worst case. Does a height-five tree's capacity of thirty-two leaves alone construct a five-comparison sorter? Why does the bound not exclude counting sort?

A03-E29 End-position placement: update one count at a time

Input is $A = [2a, 0b, 2c, 1d, 2e]$; letters identify records, and only numeric keys determine order. Keys lie in $0..2$, and output positions are one-based. Find frequencies F and cumulative end positions C. Scan right to left, place key-v records at $B[C[v]]$, then decrement $C[v]$. Record all five destination positions, full updated C, and partial outputs; explain stability.

A03-E30 Eight or sixteen bits: account for fixed costs

Sort $n \geq 1$ nonnegative 32-bit integers with LSD radix and a stable counting pass. Scheme A uses eight bits per digit; B uses sixteen. Under the explicit **cost proxy** $W = d(n + r)$, compare digit counts d, bases r, the crossover, and values at $n = 1000, 100000$. Is the crossover a measured performance threshold? With eight-byte counters, how large is each count table?

A03-E31 Match scan direction to the position convention

For $A = [2a, 0b, 2c, 1d, 2e]$, keep end positions $C = [1, 2, 5]$ and decrement after placement, but scan left to right. Trace all five placements and assess sortedness and stability. How can forward scanning be made stable by changing the position convention? Then trace both LSD passes on $[12, 11]$ to locate the failure of an unstable digit sort.

A03-E32 Squared bucket loads: where independence matters

Four independent records enter four buckets with probabilities $p = [1/2, 1/4, 1/4, 0]$. Let N_i be the load of bucket i. Use indicators to find $E[\sum_i N_i^2]$ and compare uniform buckets. Construct a correlated model with uniform marginal bucket indices and compute its squared-load sum. Does this alone prove quadratic actual insertion-sort time inside a bucket?

A03-E33 Shift negative keys and preserve query prefixes

For $[-2, 1, -2, 0, 3, 1]$, map key v to $v + 2$. Build frequency and prefix tables over $-2..3$ and count keys in closed intervals $[-1, 1]$ and $[-2, -2]$. Define boundary handling without accessing a negative index. Why preserve a separate query table when counting placement decrements counts? Give preprocessing and query costs and discuss a very large key domain.

A03-E34 Find both extrema among eleven records

There are eleven distinct keys. Initialize both extrema with the first, then pair the remaining records. Compare within each pair, then compare its smaller and larger keys with the current minimum and maximum. Count comparisons exactly, explain why initialization needs no comparison, and check $\lceil 3n/2 \rceil - 2$.

A03-E35 Local-rank conversion with one equal occurrence

Partition a current subarray of ten distinct keys into four smaller records, one equal pivot occurrence, and five larger records. For local targets $k = 7, 3, 5$, identify the retained region, its size and next local rank, or the directly returned key. Explain the subtraction of five on a right move.

A03-E36 A conservative exclusion bound for groups of five

Allow duplicate keys. Group n records by position into groups of at most five, take each lower median, then take their lower median x. Put $g = \lceil n/5 \rceil$.

Prove at least $3n/10 - 6$ records lie on each non-strict side of x, explicitly accounting for the possible incomplete group and the pivot's group. Evaluate the conservative bound at $n = 50$ and explain how it bounds a strict recursive side after three-way partitioning.

A03-E37 A noninitial subarray with duplicate keys

Use **one-based closed intervals**. Three-way partition $A[4..10] = [8, 3, 5, 3, 9, 1, 3]$ around key three. It returns $lt = 5, gt = 7$, with equal region $A[5..7]$. For local ranks $k = 1, 4, 6$, give the test and next step, and explain why comparing k directly with lt mixes different quantities.

A03-E38 The median of group medians need not be the true median

The five sorted groups are $[1, 2, 4, 17, 25]$, $[3, 5, 8, 18, 24]$, $[6, 9, 12, 19, 23]$, $[7, 10, 14, 20, 22]$, and $[11, 13, 15, 16, 21]$.

Find the median-of-medians pivot, three region sizes, next local rank when seeking rank sixteen, and the true lower median. List group-by-group certificates of nine records on each non-strict side and explain why overlap between the certificates is valid.

A03-E39 Expected larger side: apply induction to each state

With six distinct keys, pivot rank R is uniform over $1..6$. List $m_R = \max(R-1, 6-R)$ and its mean. For general n, use $\mathbb{E}[m_R] \le 3n/4$, current cost at most cn, and an inductive expected bound of a times size for every smaller state to prove linear expected time. Explain why substituting mean size into an arbitrary cost function is unjustified.

References and attribution

Reference textbook: Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition, Chapters 5–9. Supplementary notes: MIT 6.046J/18.401J, Fall 2005. Lecture 4: Quicksort (Charles E. Leiserson, 2005-09-21); Lecture 5: Sorting Lower Bounds / Linear-Time Sorting (Erik Demaine, 2005-09-26); Lecture 6: Order Statistics (Erik Demaine, 2005-09-28).

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study